

Module 2 Homework: Chapter 3 Exercises, Problems 9, 10, and 14

Ryan Waterman, 7/15/25

Problem 9

This question involves the use of multiple linear regression on the Auto data set.

This dataset was used in the 1983 American Statistical Association Exposition. It includes gas mileage, horsepower, and other information for 392 vehicles.

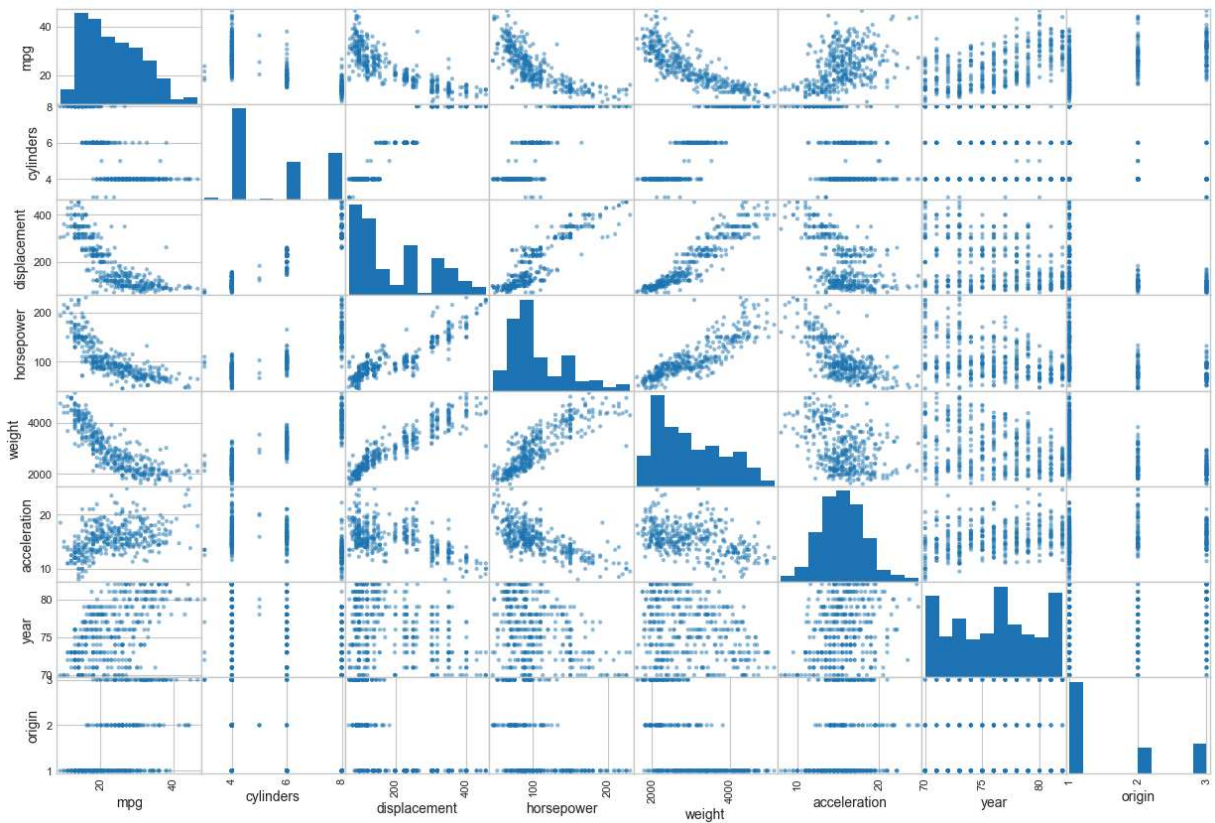
```
In [410... # Import necessary Libraries
import pandas as pd
import statsmodels.api as sm
import warnings
warnings.filterwarnings('ignore')
```

Part A

Produce a scatterplot matrix which includes all of the variables in the data set.

```
In [411... # Load the dataset and remove nulls and N/A values
auto = pd.read_csv('./data/Auto.csv')
auto.dropna(axis=0, how='any')
auto = auto[auto['horsepower'] != '?']
auto['horsepower'] = auto['horsepower'].astype('int64')

# Create a scatter matrix
pd.plotting.scatter_matrix(auto, figsize=(15,10));
```



Part B

Compute the matrix of correlations between the variables using the `DataFrame.corr()` method.

```
In [412... quantitative = auto.describe().columns
auto[quantitative].corr()
```

```
Out[412...      mpg  cylinders  displacement  horsepower  weight  acceleration
mpg      1.000000  -0.777618    -0.805127    -0.778427  -0.832244    0.423329  0.5
cylinders -0.777618    1.000000     0.950823     0.842983    0.897527   -0.504683  -0.3
displacement -0.805127  0.950823     1.000000     0.897257    0.932994   -0.543800  -0.3
horsepower -0.778427  0.842983     0.897257     1.000000    0.864538   -0.689196  -0.4
weight -0.832244  0.897527     0.932994     0.864538     1.000000   -0.416839  -0.3
acceleration 0.423329 -0.504683   -0.543800   -0.689196   -0.416839     1.000000  0.2
year 0.580541 -0.345647   -0.369855   -0.416361   -0.309120    0.290316  1.0
origin 0.565209 -0.568932   -0.614535   -0.455171   -0.585005    0.212746  0.1
```

Of this correlation matrix, the highest correlation is between displacement and cylinders, while the lowest is between year and origin.

Part C

Use the `sm.OLS()` function to perform a multiple linear regression with `mpg` as the response and all other variables except `name` as the predictors. Use the `summarize()` function to print the results. Comment on the output. For instance:

1. Is there a relationship between the predictors and the response? Use the `anova_lm()` function from `statsmodels` to answer this question.
2. Which predictors appear to have a statistically significant relationship to the response?
3. What does the coefficient for the `year` variable suggest?

Sub-Part 1

In [413...

```
y = auto['mpg']
predictors = quantitative.drop('mpg')
x = sm.add_constant(auto[predictors])
model = sm.OLS(y, x, data=auto).fit()
print(model.summary())
```

OLS Regression Results

=====						
Dep. Variable:	mpg	R-squared:	0.821			
Model:	OLS	Adj. R-squared:	0.818			
Method:	Least Squares	F-statistic:	252.4			
Date:	Fri, 18 Jul 2025	Prob (F-statistic):	2.04e-139			
Time:	17:06:05	Log-Likelihood:	-1023.5			
No. Observations:	392	AIC:	2063.			
Df Residuals:	384	BIC:	2095.			
Df Model:	7					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

const	-17.2184	4.644	-3.707	0.000	-26.350	-8.087
cylinders	-0.4934	0.323	-1.526	0.128	-1.129	0.142
displacement	0.0199	0.008	2.647	0.008	0.005	0.035
horsepower	-0.0170	0.014	-1.230	0.220	-0.044	0.010
weight	-0.0065	0.001	-9.929	0.000	-0.008	-0.005
acceleration	0.0806	0.099	0.815	0.415	-0.114	0.275
year	0.7508	0.051	14.729	0.000	0.651	0.851
origin	1.4261	0.278	5.127	0.000	0.879	1.973
=====						
Omnibus:	31.906	Durbin-Watson:	1.309			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	53.100			
Skew:	0.529	Prob(JB):	2.95e-12			
Kurtosis:	4.460	Cond. No.	8.59e+04			
=====						

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 8.59e+04. This might indicate that there are strong multicollinearity or other numerical problems.

An R^2 value of 0.821 indicates that 82.1% of the variability in the dataset can be explained by the model. The t-values associated with the year, origin, and weight predictors indicate that there is a significant relationship between these predictors and mpg. Additionally, the p-values associated with the t-statistic are less than 0.05, further supporting the claim that there is a significant relationship between these predictors and mpg. To further analyze these relationships, let's perform both type 1 and type 2 ANOVA tests.

Sub-Part 2

In [414... `from ISLP.models import ModelSpec as MS`

Type 1 ANOVA

In [415... `# LINK TO REFERENCE: https://github.com/statsmodels/statsmodels/issues/8241
design = MS(predictors)
THIS WAS FOUND HERE: https://islp.readthedocs.io/en/latest/models/spec.html`


```

design.fit_transform(auto)
anova = sm.stats.anova_lm(*[sm.OLS(y, D).fit() for D in design.build_sequence(auto,
anova.index = design.names

anova

```

Out[415...

	df_resid	ssr	df_diff	ss_diff	F	Pr(>F)
intercept	391.0	23818.993469	0.0	NaN	NaN	NaN
cylinders	390.0	9415.910391	1.0	14403.083079	1300.683788	2.806569e-126
displacement	389.0	8342.566366	1.0	1073.344025	96.929329	1.442524e-20
horsepower	388.0	7939.158297	1.0	403.408069	36.430140	3.700154e-09
weight	387.0	6963.433344	1.0	975.724953	88.113748	5.378297e-19
acceleration	386.0	6962.467273	1.0	0.966071	0.087242	7.678720e-01
year	385.0	4543.347025	1.0	2419.120249	218.460900	1.794078e-39
origin	384.0	4252.212530	1.0	291.134494	26.291171	4.665681e-07

The type 1 ANOVA sequentially steps through the predictors and compares the response of each new predictor in the sequence to the previous. This is not helpful for determining the relationship between the response and predictors, as the order of the predictors in the sequence could skew their resulting response indication. A Type 2 ANOVA will be better suited for this application, as it is not dependent on the order of the predictors and assumes the predictor variables have no interaction. This will explicitly identify the predictors that have the most effect on the response.

Type 2 ANOVA

In [416...

```

# CODE REFERENCE FOUND HERE: https://islp.readthedocs.io/en/latest/models/anova.htm
dfs = []
for d in design.build_sequence(auto, anova_type='drop'):
    dfs.append(sm.stats.anova_lm(sm.OLS(y,d).fit(), model).iloc[1:])
df = pd.concat(dfs)
df.index = design.names
df

```

Out[416...

	df_resid	ssr	df_diff	ss_diff	F	Pr(>F)
intercept	384.0	4252.21253	1.0	152.205951	13.745100	2.401841e-04
cylinders	384.0	4252.21253	1.0	25.791491	2.329125	1.277965e-01
displacement	384.0	4252.21253	1.0	77.612668	7.008884	8.444649e-03
horsepower	384.0	4252.21253	1.0	16.739754	1.511699	2.196328e-01
weight	384.0	4252.21253	1.0	1091.631693	98.580813	7.874953e-21
acceleration	384.0	4252.21253	1.0	7.358417	0.664509	4.154780e-01
year	384.0	4252.21253	1.0	2402.249906	216.937408	3.055983e-39
origin	384.0	4252.21253	1.0	291.134494	26.291171	4.665681e-07

Based on the `ss_diff` values of 1091 and 2402 for weight and year predictors, respectively, it is clear that these predictors have the most influence on the response. Additionally, these predictors have statistically significant p-values associated with the F statistic. Statistical significance is true for the origin predictor as well, although the resulting `ss_diff` is not as significant as weight and year.

Sub-Part 3

The year coefficient suggests that for each unit increase in year, mpg increases proportionally by 0.7508.

Part D

Produce some of diagnostic plots of the linear regression fit as described in the lab. Comment on any problems you see with the fit. Do the residual plots suggest any unusually large outliers? Does the leverage plot identify any observations with unusually high leverage?

In [417...

```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
```

In [418...

```
# RESIDUALS PLOT REFERENCE LINK: https://seaborn.pydata.org/generated/seaborn.resid

residuals = model.resid
fitted_values = model.fittedvalues

# Set up the plot
plt.style.use('seaborn-v0_8-whitegrid')
fig, ax = plt.subplots(figsize=(10, 7))

# Use seaborn.residplot which creates the scatter plot and adds a LOWESS smoother
```

```

sns.residplot(
    x=fitted_values,
    y=residuals,
    lowess=True, # This adds the LOWESS trend line
    ax=ax,
    scatter_kws={'alpha': 0.5, 'facecolors': 'lightgray', 'edgecolors': 'k'},
    line_kws={'color': 'red', 'lw': 2}
)

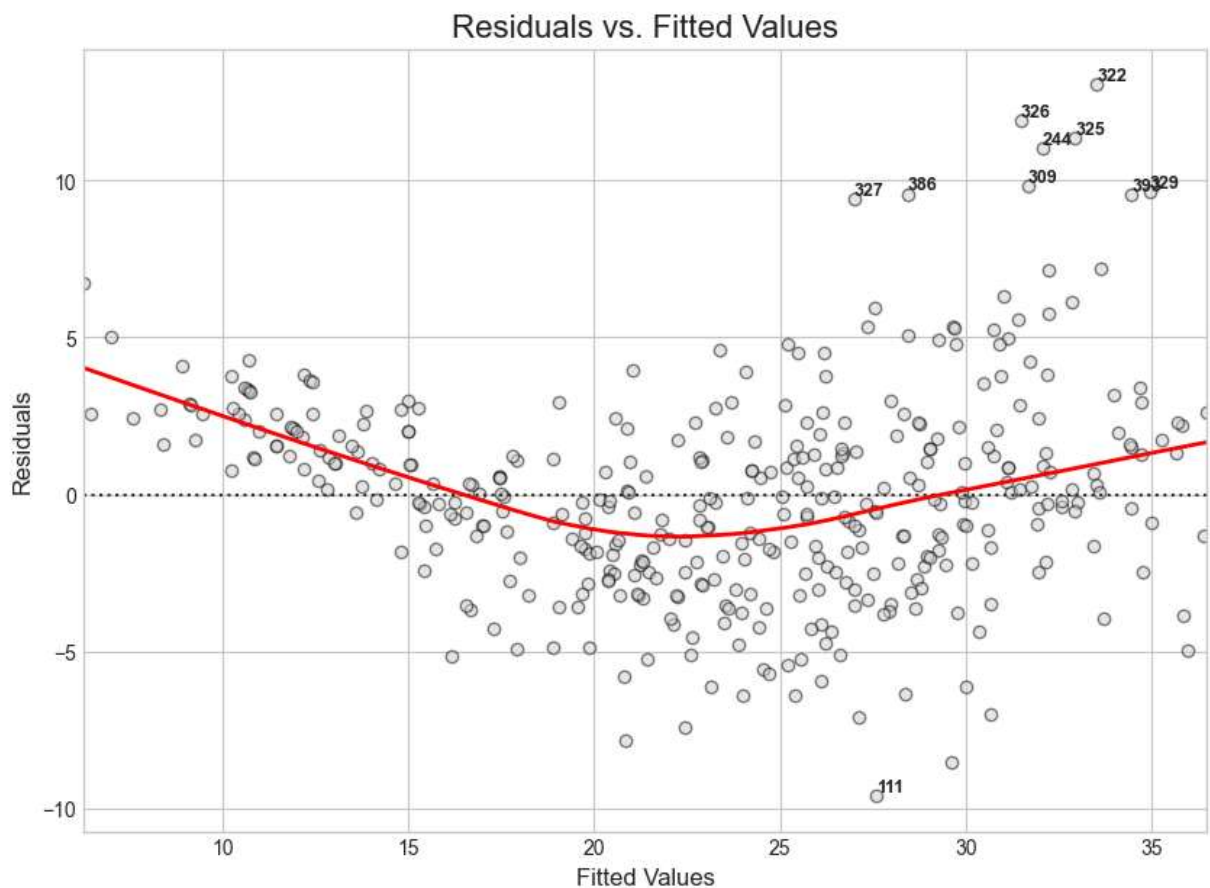
#Identify the outlying points, reference found here: https://stackoverflow.com/ques
abs_residuals = np.absolute(residuals)
top_n = 10
top_indices = abs_residuals.nlargest(top_n).index

# Add text labels for these points
for i in top_indices:
    ax.text(fitted_values[i], residuals[i], str(i),
            horizontalalignment='left',
            verticalalignment='bottom',
            fontsize=9,
            fontweight='bold')

# Set titles and labels for clarity
ax.set_title('Residuals vs. Fitted Values', fontsize=16)
ax.set_xlabel('Fitted Values', fontsize=12)
ax.set_ylabel('Residuals', fontsize=12)

plt.show()

```



This plot exhibits heteroscedasticity, which can be corrected with a log transform. Let's do that.

In [419...

```

y_log = np.log(auto['mpg'])
predictors = quantitative.drop('mpg')
log_corrected_auto = auto.copy()
for pred in predictors:
    log_corrected_auto[pred] = np.log(log_corrected_auto[pred])
x_log = sm.add_constant(log_corrected_auto[predictors])
model_log = sm.OLS(y_log, x_log, data=log_corrected_auto).fit()
print(model_log.summary())

```

OLS Regression Results

```

=====
Dep. Variable:          mpg      R-squared:                0.890
Model:                  OLS      Adj. R-squared:           0.888
Method:                 Least Squares      F-statistic:         445.3
Date:                  Fri, 18 Jul 2025      Prob (F-statistic):    5.86e-180
Time:                  17:06:05      Log-Likelihood:       300.34
No. Observations:      392      AIC:                  -584.7
Df Residuals:          384      BIC:                  -552.9
Df Model:               7
Covariance Type:       nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	-0.1554	0.648	-0.240	0.811	-1.430	1.119
cylinders	-0.0828	0.061	-1.348	0.178	-0.204	0.038
displacement	0.0066	0.057	0.116	0.907	-0.105	0.119
horsepower	-0.2944	0.058	-5.106	0.000	-0.408	-0.181
weight	-0.5697	0.082	-6.914	0.000	-0.732	-0.408
acceleration	-0.1792	0.060	-3.011	0.003	-0.296	-0.062
year	2.2440	0.132	17.044	0.000	1.985	2.503
origin	0.0448	0.019	2.383	0.018	0.008	0.082

```

=====
Omnibus:                8.794      Durbin-Watson:           1.501
Prob(Omnibus):           0.012      Jarque-Bera (JB):        14.713
Skew:                   -0.059      Prob(JB):                0.000638
Kurtosis:                3.942      Cond. No.                 1.36e+03
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 1.36e+03. This might indicate that there are strong multicollinearity or other numerical problems.

In [420...

```

# RESIDUALS PLOT REFERENCE LINK: https://seaborn.pydata.org/generated/seaborn.resid

residuals_log = model_log.resid
fitted_values_log = model_log.fittedvalues

# Set up the plot
plt.style.use('seaborn-v0_8-whitegrid')
fig, ax = plt.subplots(figsize=(10, 7))

```

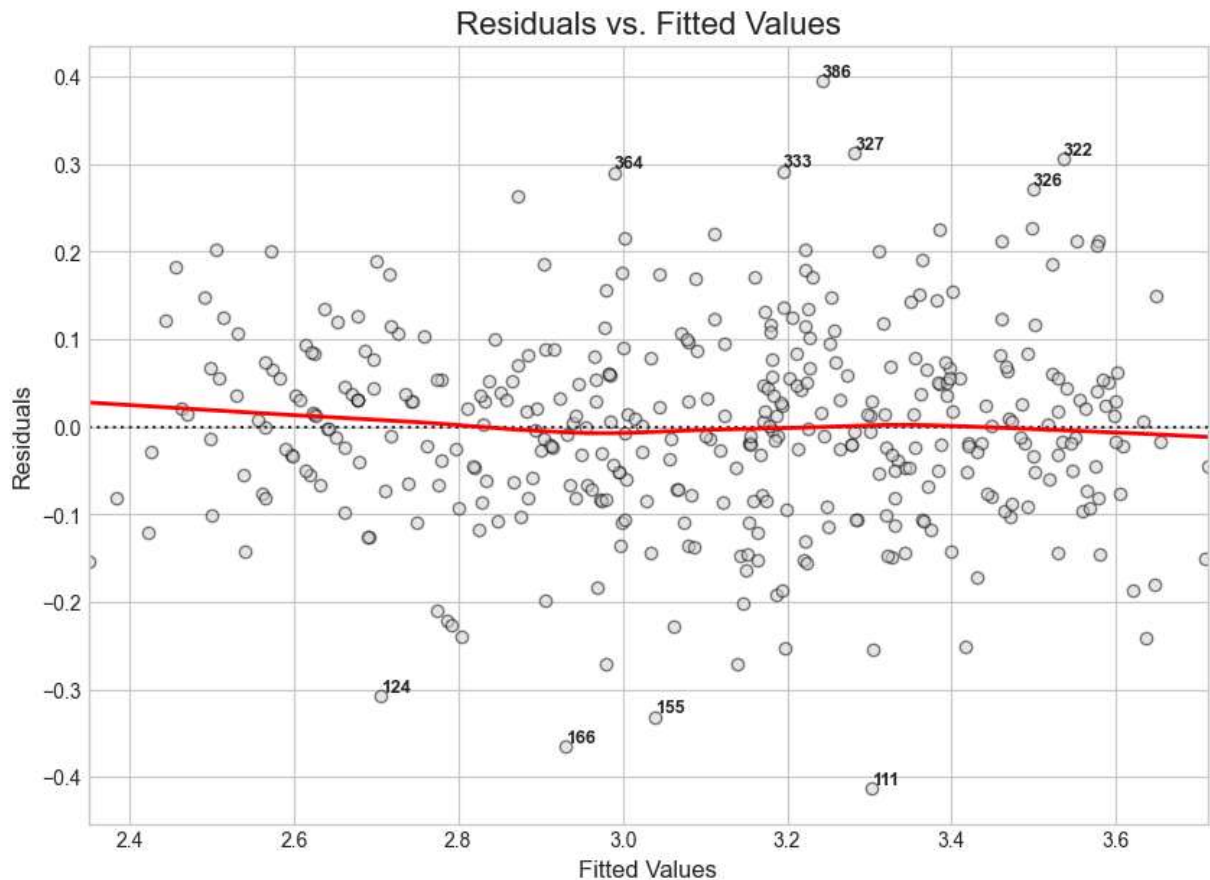
```
# Use seaborn.residplot which creates the scatter plot and adds a LOWESS smoother
sns.residplot(
    x=fitted_values_log,
    y=residuals_log,
    lowess=True, # This adds the LOWESS trend line
    ax=ax,
    scatter_kws={'alpha': 0.5, 'facecolors': 'lightgray', 'edgecolors': 'k'},
    line_kws={'color': 'red', 'lw': 2}
)

#Identify the outlying points, reference found here: https://stackoverflow.com/questions/11554253/how-to-find-outliers-in-a-scatter-plot
abs_residuals_log = np.absolute(residuals_log)
top_n = 10
top_indices = abs_residuals_log.nlargest(top_n).index

# Add text labels for these points
for i in top_indices:
    ax.text(fitted_values_log[i], residuals_log[i], str(i),
            horizontalalignment='left',
            verticalalignment='bottom',
            fontsize=9,
            fontweight='bold')

# Set titles and labels for clarity
ax.set_title('Residuals vs. Fitted Values', fontsize=16)
ax.set_xlabel('Fitted Values', fontsize=12)
ax.set_ylabel('Residuals', fontsize=12)

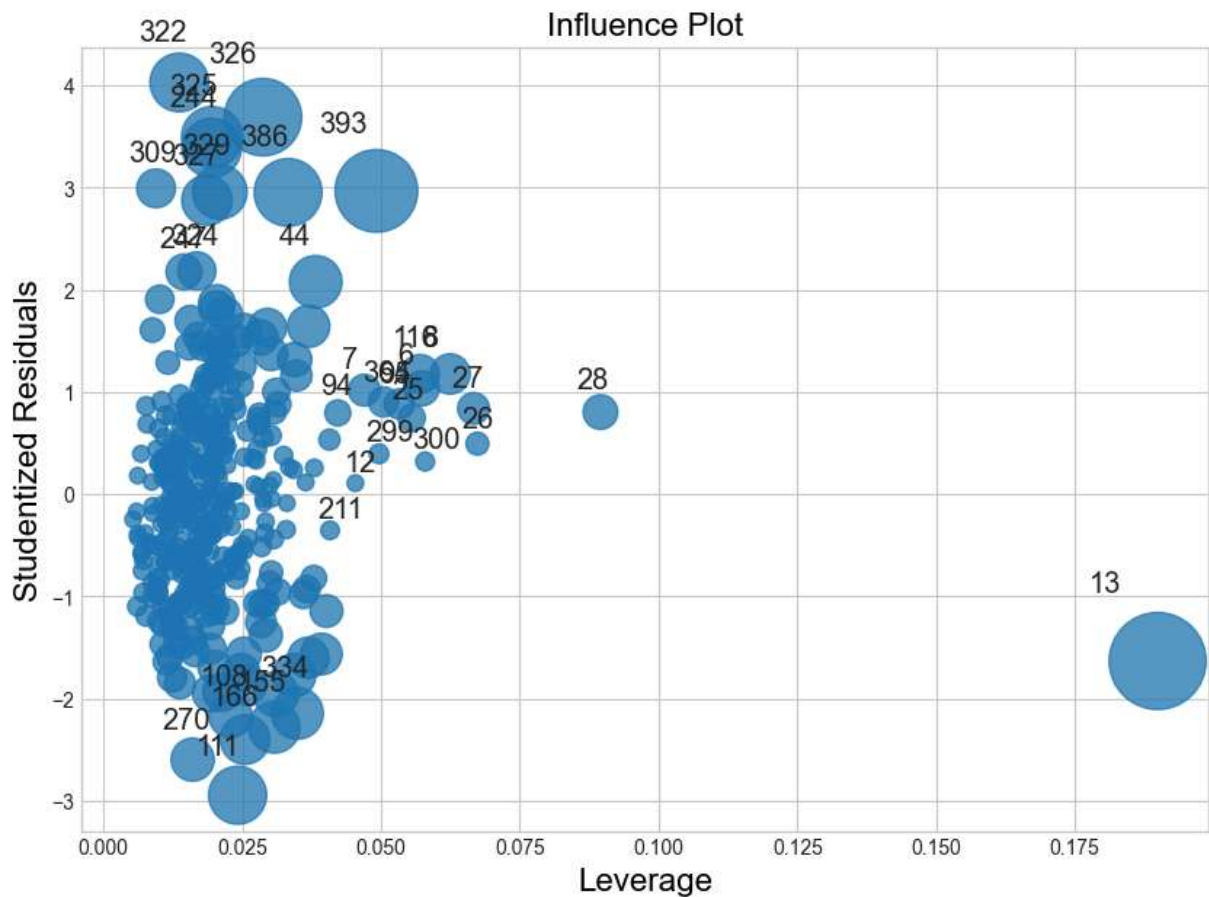
plt.show()
```



Great, the heteroscedasticity has been corrected. The outliers seem to have changed, but this makes sense given the non-linear effect of the log transform. Data points 326 and 327 were consistently outliers in both normal and log transformed residual plots. The key outcome of the residual plots is that the model is not properly capturing some non-linear relationships in the data, which, looking back at the scatter matrix, is expected. Weight, Horsepower, Displacement, and Acceleration are all non-linearly related to mpg, so they are likely contributing to the effect seen in the residuals plots.

Now, let's look at the leverage plot.

```
In [421... fig, ax = plt.subplots(figsize=(10, 7))
fig = sm.graphics.influence_plot(model, ax=ax, criterion="cooks")
```

Based on this leverage plot, which utilizes Cook's for bubble scaling, data point 13 is the most influential in the dataset. Interestingly, this point is not an outlier. It is an unusual point, but fits the model's trend well. All of the outliers identified in the residuals plots exhibit high Cook's distances, which I would expect because outliers have a disproportionate influence on the model. However, their leverage is low, implying that the model does not predict these values well, therefore they are true outliers.

Part E

Fit some models with interactions as described in the lab. Do any interactions appear to be statistically significant?

In [422... `import statsmodels.formula.api as smf`

I am interested in the following interactions of predictors on mpg:

1. Year and Weight
2. Year and Origin
3. Acceleration and Displacement
4. Horsepower and Weight

```
In [423... year_weight_model = smf.ols('mpg ~ year * weight', data=auto).fit()
year_origin_model = smf.ols('mpg ~ year * origin', data=auto).fit()
accel_disp_model = smf.ols('mpg ~ acceleration * displacement', data=auto).fit()
hp_weight_model = smf.ols('mpg ~ horsepower * weight', data=auto).fit()
```

```
In [424... # Source found here: https://stackoverflow.com/questions/51734180/converting-statsmodels
year_weight_model_results = pd.read_html(year_weight_model.summary().tables[1]).as_html()
print(f'R-Squared: {round(year_weight_model.rsquared, 3)}')
print(year_weight_model_results)
```

R-Squared: 0.834

	coef	std err	t	P> t	[0.025	0.975]
Intercept	-110.4519	12.947000	-8.531	0.0	-135.907	-84.997
year	2.0405	0.172000	11.876	0.0	1.703	2.378
weight	0.0275	0.004000	6.242	0.0	0.019	0.036
year:weight	-0.0005	0.000059	-7.752	0.0	-0.001	-0.000

```
In [425... year_origin_model_results = pd.read_html(year_origin_model.summary().tables[1]).as_html()
print(f'R-Squared: {round(year_origin_model.rsquared, 3)}')
print(year_origin_model_results)
```

R-Squared: 0.56

	coef	std err	t	P> t	[0.025	0.975]
Intercept	-83.3809	12.000	-6.948	0.000	-106.974	-59.788
year	1.3089	0.158	8.305	0.000	0.999	1.619
origin	17.3752	6.833	2.543	0.011	3.942	30.809
year:origin	-0.1663	0.089	-1.871	0.062	-0.341	0.008

```
In [426... accel_disp_model_results = pd.read_html(accel_disp_model.summary().tables[1]).as_html()
print(f'R-Squared: {round(accel_disp_model.rsquared, 3)}')
print(accel_disp_model_results)
```

R-Squared: 0.677

	coef	std err	t	P> t	[0.025	0.975]
Intercept	23.0533	2.922	7.889	0.000	17.308	28.798
acceleration	0.8303	0.182	4.574	0.000	0.473	1.187
displacement	0.0031	0.011	0.277	0.782	-0.019	0.025
acceleration:displacement	-0.0046	0.001	-5.799	0.000	-0.006	-0.003

```
In [427... hp_weight_model_results = pd.read_html(hp_weight_model.summary().tables[1]).as_html()
print(f'R-Squared: {round(hp_weight_model.rsquared, 3)}')
print(hp_weight_model_results)
```

R-Squared: 0.748

	coef	std err	t	P> t	[0.025	0.975]
Intercept	63.557900	2.343000	27.127	0.0	58.951000	68.164000
horsepower	-0.250800	0.027000	-9.195	0.0	-0.304000	-0.197000
weight	-0.010800	0.001000	-13.921	0.0	-0.012000	-0.009000
horsepower:weight	0.000054	0.000007	8.054	0.0	0.000041	0.000067

After analyzing each of the interactions, I have concluded the following interactions are statistically significant when predicting mpg:

1. **Year:Weight**
2. **Acceleration:Displacement**
3. **Horsepower:Weight**

Each of these interactions have a p-value of less than 0.05, indicating that they are statistically significant.

Just out of curiosity, I'd like to look at the residual plots for these models. I suspect some of the relationships will be quite linear.

```
In [428... def plot_residuals(model, ax, title, x_label, y_label, outlier_count = 10):
    # RESIDUALS PLOT REFERENCE LINK: https://seaborn.pydata.org/generated/seaborn.r
    residuals = model.resid
    fitted_values = model.fittedvalues

    # Use seaborn.residplot which creates the scatter plot and adds a LOWESS smooth
    sns.residplot(
        x=fitted_values,
        y=residuals,
        lowess=True, # This adds the LOWESS trend line
        ax=ax,
        scatter_kws={'alpha': 0.5, 'facecolors': 'lightgray', 'edgecolors': 'k'},
        line_kws={'color': 'red', 'lw': 2}
    )

    #Identify the outlying points, reference found here: https://stackoverflow.com/
    abs_residuals = np.absolute(residuals)
    top_n = outlier_count
    top_indices = abs_residuals.nlargest(top_n).index

    # Add text labels for these points
    for i in top_indices:
        ax.text(fitted_values[i], residuals[i], str(i),
                horizontalalignment='left',
                verticalalignment='bottom',
                fontsize=9,
                fontweight='bold')

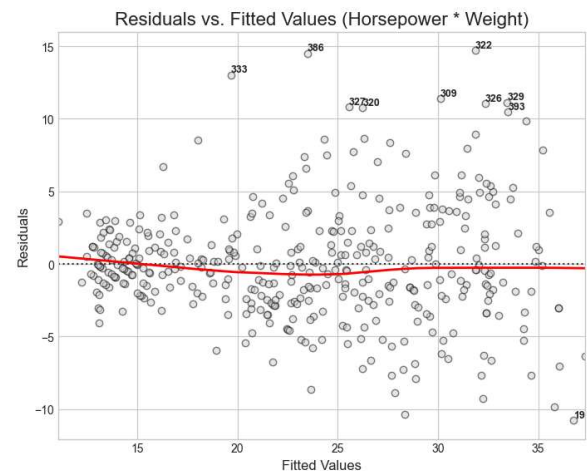
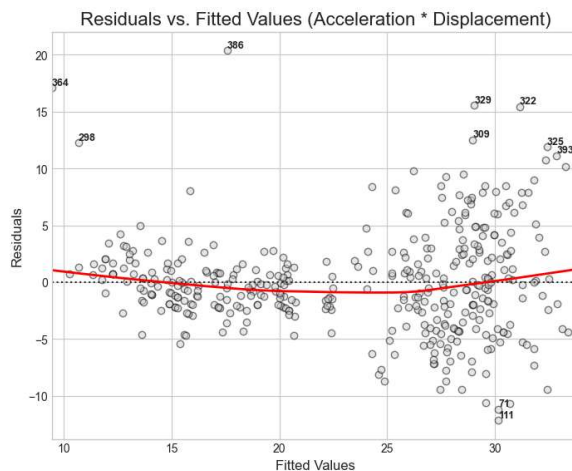
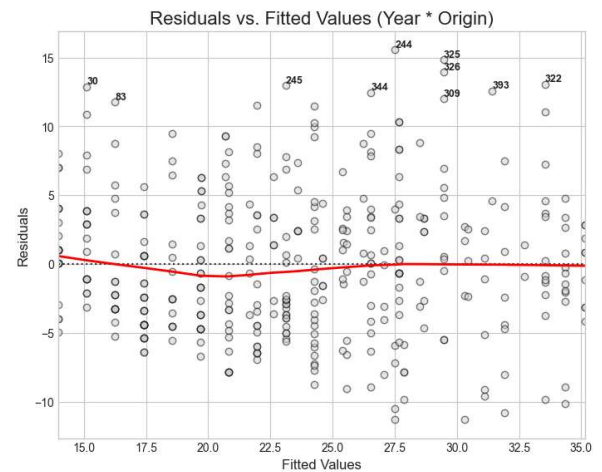
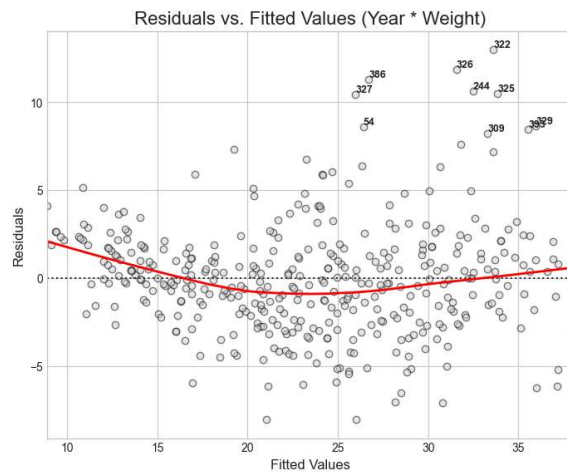
    # Set titles and labels for clarity
    ax.set_title(title, fontsize=16)
    ax.set_xlabel(x_label, fontsize=12)
    ax.set_ylabel(y_label, fontsize=12)
```

```
In [429... fig, axes = plt.subplots(2, 2, figsize=(18,14));

title='Residuals vs. Fitted Values'
x_label='Fitted Values'
y_label='Residuals'

plot_residuals(year_weight_model, axes[0,0], title=f'{title} (Year * Weight)', x_la
plot_residuals(year_origin_model, axes[0,1], title=f'{title} (Year * Origin)', x_la
plot_residuals(accel_disp_model, axes[1,0], title=f'{title} (Acceleration * Displac
plot_residuals(hp_weight_model, axes[1,1], title=f'{title} (Horsepower * Weight)',

plt.show()
```



These residual plots are far more linear than what was seen with the model that encompassed all predictors, with the exception of Year:Weight. Based on the residual plots and R^2 values, Horsepower:Weight is the most linear relationship with the most accurate model.

Part F

Try a few different transformations of the variables, such as $\log(X)$, $\sqrt{X}$, X^2 . Comment on your findings.

The log transformed model was completed in a previous part. Let's do the other transforms.

In [430...

```
y_sqrt = np.sqrt(auto['mpg'])
sqrt_transformed_auto = auto.copy()

for pred in predictors:
    sqrt_transformed_auto[pred] = np.sqrt(sqrt_transformed_auto[pred])

x_sqrt = sm.add_constant(sqrt_transformed_auto[predictors])
model_sqrt = sm.OLS(y_sqrt, x_sqrt, data=sqrt_transformed_auto).fit()
```

```
In [431... y_square = np.square(auto['mpg'])
square_transformed_auto = auto.copy()

for pred in predictors:
    square_transformed_auto[pred] = np.square(square_transformed_auto[pred])

x_square = sm.add_constant(square_transformed_auto[predictors])
model_square = sm.OLS(y_square, x_square, data=square_transformed_auto).fit()
```

```
In [432... model_log_results = pd.read_html(model_log.summary().tables[1].as_html(), header=0,
print(f'R-Squared: {round(model_log.rsquared, 3)}')
print(model_log_results)
```

R-Squared: 0.89

	coef	std err	t	P> t	[0.025	0.975]
const	-0.1554	0.648	-0.240	0.811	-1.430	1.119
cylinders	-0.0828	0.061	-1.348	0.178	-0.204	0.038
displacement	0.0066	0.057	0.116	0.907	-0.105	0.119
horsepower	-0.2944	0.058	-5.106	0.000	-0.408	-0.181
weight	-0.5697	0.082	-6.914	0.000	-0.732	-0.408
acceleration	-0.1792	0.060	-3.011	0.003	-0.296	-0.062
year	2.2440	0.132	17.044	0.000	1.985	2.503
origin	0.0448	0.019	2.383	0.018	0.008	0.082

```
In [433... model_sqrt_results = pd.read_html(model_sqrt.summary().tables[1].as_html(), header=
print(f'R-Squared: {round(model_sqrt.rsquared, 3)}')
print(model_sqrt_results)
```

R-Squared: 0.866

	coef	std err	t	P> t	[0.025	0.975]
const	-1.9493	0.847	-2.300	0.022	-3.616	-0.283
cylinders	-0.1086	0.142	-0.765	0.445	-0.388	0.171
displacement	0.0197	0.021	0.930	0.353	-0.022	0.061
horsepower	-0.0909	0.028	-3.197	0.002	-0.147	-0.035
weight	-0.0614	0.007	-8.422	0.000	-0.076	-0.047
acceleration	-0.1073	0.077	-1.392	0.165	-0.259	0.044
year	1.2660	0.079	15.963	0.000	1.110	1.422
origin	0.2723	0.071	3.842	0.000	0.133	0.412

```
In [434... model_square_results = pd.read_html(model_square.summary().tables[1].as_html(), hea
print(f'R-Squared: {round(model_square.rsquared, 3)}')
print(model_square_results)
```

R-Squared: 0.706

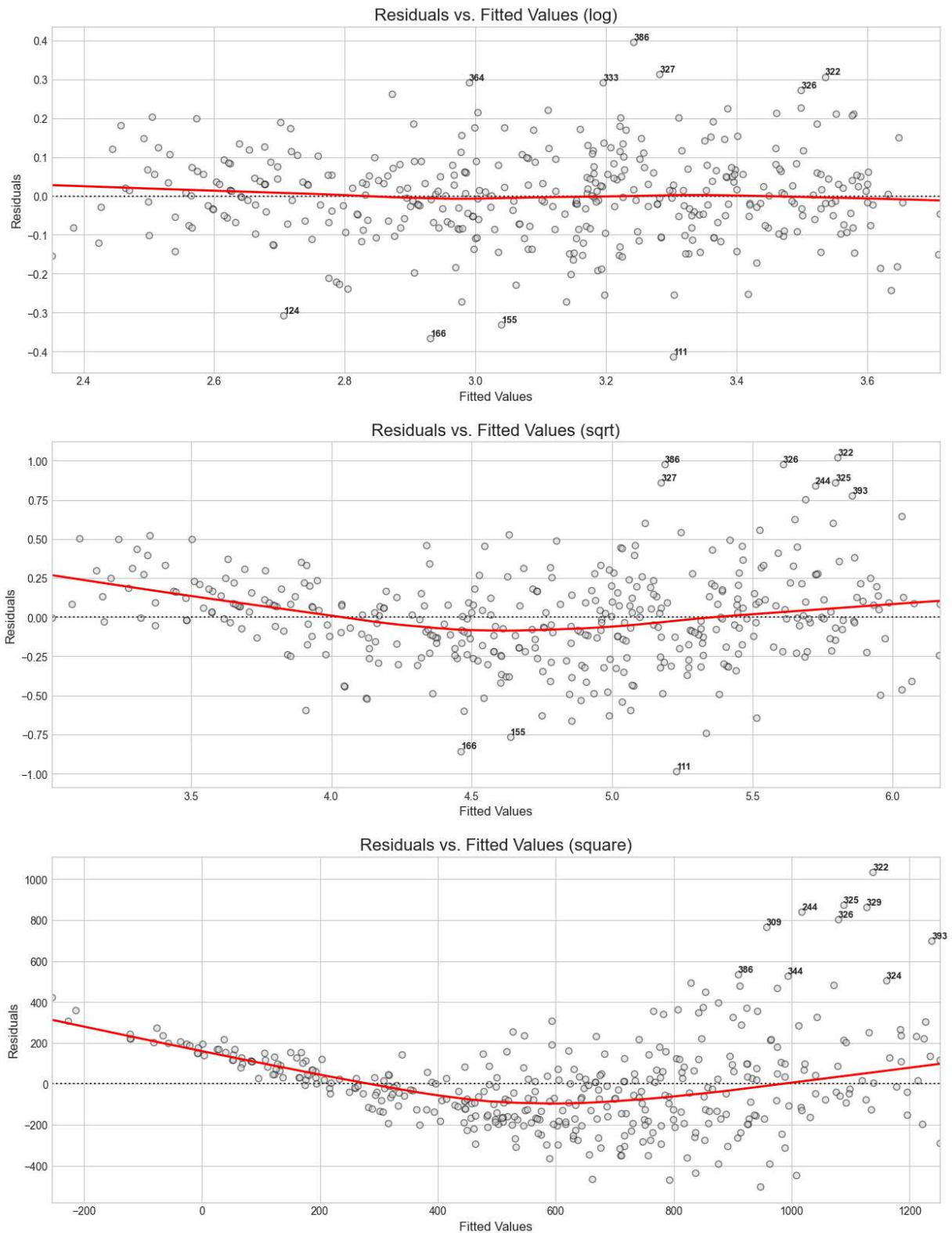
	coef	std err	t	P> t	[0.025	0.975]
const	-752.288000	145.649000	-5.165	0.000	-1038.658000	-465.918000
cylinders	-3.746000	1.559000	-2.403	0.017	-6.810000	-0.682000
displacement	0.003400	0.001000	3.926	0.000	0.002000	0.005000
horsepower	0.000100	0.003000	0.042	0.967	-0.006000	0.006000
weight	-0.000048	0.000006	-8.707	0.000	-0.000059	-0.000037
acceleration	0.489200	0.166000	2.941	0.003	0.162000	0.816000
year	0.273100	0.022000	12.513	0.000	0.230000	0.316000
origin	26.083900	4.275000	6.101	0.000	17.678000	34.489000

```
In [435... fig, axes = plt.subplots(3,1, figsize=(15,20));
```

```
title='Residuals vs. Fitted Values'
x_label='Fitted Values'
y_label='Residuals'

plot_residuals(model_log, axes[0], title=f'{title} (log)', x_label=x_label, y_label=y_label)
plot_residuals(model_sqrt, axes[1], title=f'{title} (sqrt)', x_label=x_label, y_label=y_label)
plot_residuals(model_square, axes[2], title=f'{title} (square)', x_label=x_label, y_label=y_label)

plt.show()
```

These transforms to the data impacted the models as expected. The log transform 'linearized' the data, resulting in a higher R^2 value and more statistically significant predictors. The square root transform worked to remove some of the non-linearity from the data, however, it is clear that there are some polynomial relationships that could not be corrected with a square-root transformation. This is evident from the non-linear residual plot. Lastly, the square transform produced the worst R^2 value,

which was expected. This transform would exacerbate any interactions and non-linear behavior, and OLS is a linear model. Interestingly, in the square transform model, acceleration and cylinders became statistically significant predictors of mpg.

Problem 10

This question should be answered using the Carseats data set.

Carseats is simulated data set containing sales of child car seats at 400 different stores.

In [436...

```
from ISLP import load_data

carseats = load_data('carseats')
carseats.head(5)
```

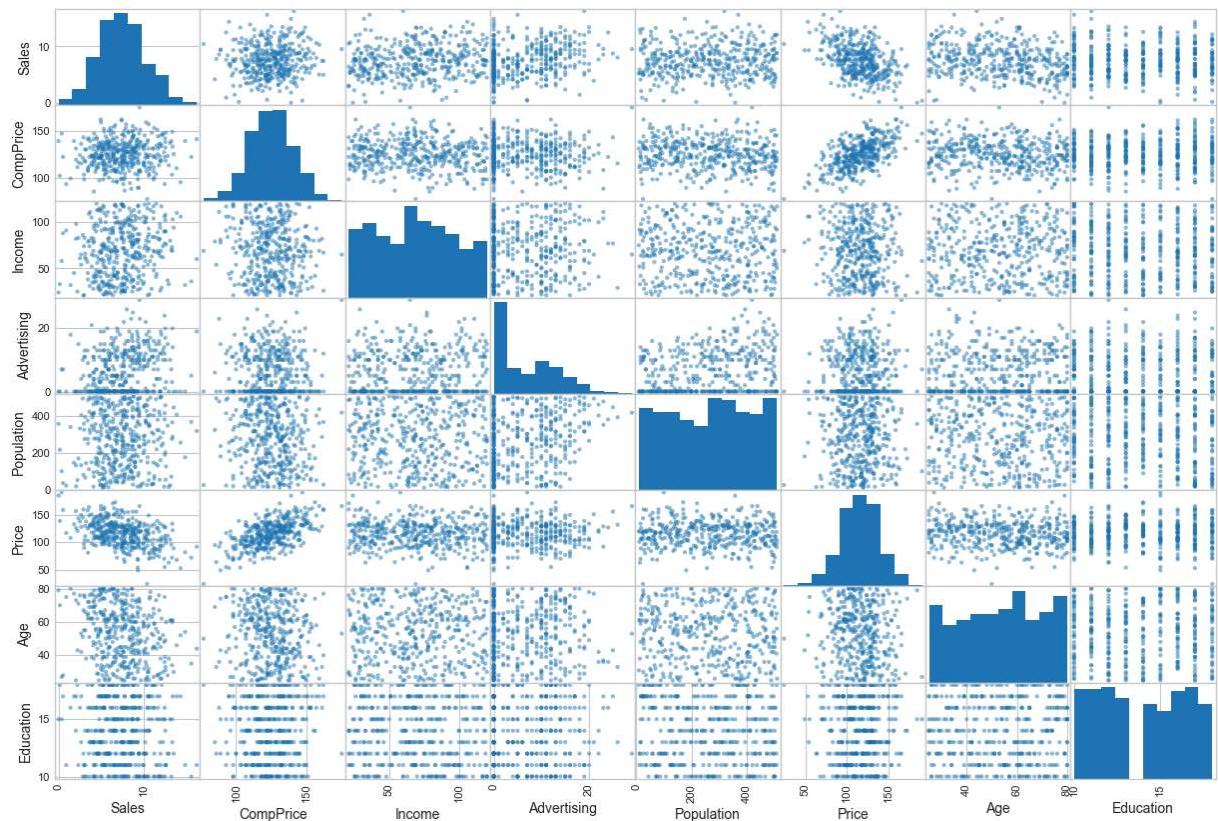
Out[436...

	Sales	CompPrice	Income	Advertising	Population	Price	ShelveLoc	Age	Education
0	9.50	138	73	11	276	120	Bad	42	17
1	11.22	111	48	16	260	83	Good	65	10
2	10.06	113	35	10	269	80	Medium	59	12
3	7.40	117	100	4	466	97	Medium	55	14
4	4.15	141	64	3	340	128	Bad	38	13



In [437...

```
pd.plotting.scatter_matrix(carseats, figsize=(15,10));
```



Part A

Fit a multiple regression model to predict Sales using Price, Urban, and US.

Urban and US are qualitative measures. To address this for the linear model, I am going to remap their values to 0 (No) and 1 (Yes).

```
In [438... carseats_remapped = carseats.copy()
carseats_remapped['Urban'].replace({'Yes':1, 'No':0}, inplace=True)
carseats_remapped['US'].replace({'Yes':1, 'No':0}, inplace=True)
carseats_remapped.head(5)
```

```
Out[438...   Sales  CompPrice  Income  Advertising  Population  Price  ShelfLoc  Age  Education
0    9.50         138      73           11          276    120       Bad   42           17
1   11.22         111      48           16          260     83       Good   65           10
2   10.06         113      35           10          269     80      Medium   59           12
3    7.40         117     100            4          466     97      Medium   55           14
4    4.15         141      64            3          340    128       Bad   38           13
```

```
In [439... y_carseat = carseats_remapped['Sales']
predictors = ['Price', 'Urban', 'US']
```

```
x_carseat = sm.add_constant(carseats_remapped[predictors])
carseat_remapped_model = sm.OLS(y_carseat,x_carseat,data=carseats_remapped).fit()
print(carseat_remapped_model.summary())
```

OLS Regression Results

```
=====
Dep. Variable:          Sales    R-squared:                0.239
Model:                  OLS      Adj. R-squared:           0.234
Method:                 Least Squares    F-statistic:          41.52
Date:                  Fri, 18 Jul 2025    Prob (F-statistic):    2.39e-23
Time:                  17:06:09    Log-Likelihood:        -927.66
No. Observations:      400      AIC:                  1863.
Df Residuals:          396      BIC:                  1879.
Df Model:               3
Covariance Type:       nonrobust
=====
```

	coef	std err	t	P> t	[0.025	0.975]
const	13.0435	0.651	20.036	0.000	11.764	14.323
Price	-0.0545	0.005	-10.389	0.000	-0.065	-0.044
Urban	-0.0219	0.272	-0.081	0.936	-0.556	0.512
US	1.2006	0.259	4.635	0.000	0.691	1.710

```
=====
Omnibus:                0.676    Durbin-Watson:           1.912
Prob(Omnibus):           0.713    Jarque-Bera (JB):         0.758
Skew:                    0.093    Prob(JB):                 0.684
Kurtosis:                2.897    Cond. No.                  628.
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Now, I want to do the same thing leaving the qualitative measures untouched. I suspect there will be a difference in the model summary.

In [440...

```
carseat_model = smf.ols('Sales ~ Price + Urban + US', data=carseats).fit()
print(carseat_model.summary())
```

```

=====
                        OLS Regression Results
=====
Dep. Variable:          Sales    R-squared:                0.239
Model:                  OLS      Adj. R-squared:           0.234
Method:                 Least Squares    F-statistic:             41.52
Date:                  Fri, 18 Jul 2025    Prob (F-statistic):      2.39e-23
Time:                  17:06:09    Log-Likelihood:         -927.66
No. Observations:      400      AIC:                    1863.
Df Residuals:          396      BIC:                    1879.
Df Model:              3
Covariance Type:       nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
Intercept	13.0435	0.651	20.036	0.000	11.764	14.323
Urban[T.Yes]	-0.0219	0.272	-0.081	0.936	-0.556	0.512
US[T.Yes]	1.2006	0.259	4.635	0.000	0.691	1.710
Price	-0.0545	0.005	-10.389	0.000	-0.065	-0.044

```

=====
Omnibus:                0.676    Durbin-Watson:           1.912
Prob(Omnibus):          0.713    Jarque-Bera (JB):        0.758
Skew:                   0.093    Prob(JB):                0.684
Kurtosis:               2.897    Cond. No.                 628.
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Okay, so the coefficient output is the same. The second OLS model must automatically remap qualitative variables... Interesting.

Part B

Provide an interpretation of each coefficient in the model. Be careful—some of the variables in the model are qualitative!

1. Urban

- Holding price and US location constant, an urban store is predicted to have sales that are, on average, 0.0219 units lower than a non-urban store. The coefficient is a per unit measure relative to Sales, therefore the average difference between urban and non-urban sales is 0.0219 per unit.

2. US

- Similar to Urban, a coefficient of 1.2006 indicates a unit increase of 1.2006 Sales for a 'Yes' value in the US dataset, relative to a baseline of 'No.'

3. Price

- A price coefficient of -0.0545 indicates that a unit price change results in 0.0545 fewer sales.

Part C

Write out the model in equation form, being careful to handle the qualitative variables properly.

Base MLR Formula:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_k x_k + \varepsilon$$

The Urban and US variables were evaluated as binary values of 1 or 0, the equation can be modeled as seen, below:

$$Y = 13.0435 - (0.0219 * \text{Urban}[\text{Yes}]) + (1.2006 * \text{US}[\text{Yes}]) - 0.0545 * \text{Price} + \varepsilon$$

$$\rightarrow Y = 13.0435 - (0.0219 * 1) + (1.2006 * 1) - 0.0545 * \text{Price} + \varepsilon$$

This model is generally true for both Yes and No cases in Urban and US because the terms drop out when they are evaluated as No, resulting in the equation, below:

$$Y = 13.0435 - (0.0219 * \text{Urban}[\text{No}]) + (1.2006 * \text{US}[\text{No}]) - 0.0545 * \text{Price} + \varepsilon$$

$$\rightarrow Y = 13.0435 - (0.0219 * 0) + (1.2006 * 0) - 0.0545 * \text{Price} + \varepsilon$$

$$\rightarrow Y = 13.0435 - 0.0545 * \text{Price} + \varepsilon$$

NOTE: This is also true for any combination of Yes and No for Urban or US. The terms will always drop out if they are No.

Part D

For which of the predictors can you reject the null hypothesis $H_0: \beta_j = 0$?

The p-value associated with the t-statistic can be used to determine which predictors are statistically significant (i.e. the null hypothesis is rejected). Based on a significance level of 0.05, US[T.Yes] and Price are statistically significant because their p-values are 0.

Part E

On the basis of your response to the previous question, fit a smaller model that only uses the predictors for which there is evidence of association with the outcome.

```
In [441... smaller_carseat_model = smf.ols('Sales ~ Price + US', data=carseats).fit()
print(smaller_carseat_model.summary())
```



```

=====
                        OLS Regression Results
=====
Dep. Variable:          Sales    R-squared:                0.239
Model:                  OLS      Adj. R-squared:           0.235
Method:                 Least Squares    F-statistic:             62.43
Date:                   Fri, 18 Jul 2025    Prob (F-statistic):      2.66e-24
Time:                   17:06:09    Log-Likelihood:         -927.66
No. Observations:       400      AIC:                    1861.
Df Residuals:           397      BIC:                    1873.
Df Model:                2
Covariance Type:        nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
Intercept	13.0308	0.631	20.652	0.000	11.790	14.271
US[T.Yes]	1.1996	0.258	4.641	0.000	0.692	1.708
Price	-0.0545	0.005	-10.416	0.000	-0.065	-0.044

```

=====
Omnibus:                0.666    Durbin-Watson:           1.912
Prob(Omnibus):          0.717    Jarque-Bera (JB):        0.749
Skew:                   0.092    Prob(JB):                0.688
Kurtosis:               2.895    Cond. No.:               607.
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Part F

How well do the models in (a) and (e) fit the data?

Both models have an R^2 value of 0.239, suggesting they equally fit the model. An R^2 value of 0.239 indicates that the model only explains 23.9% of the variance in the data, which is poor. The models do not fit the data well.

Part G

Using the model from (e), obtain 95% confidence intervals for the coefficient(s).

```
In [442... smaller_carseat_model.conf_int(alpha=0.05)
```

```
Out[442...

```

	0	1
Intercept	11.79032	14.271265
US[T.Yes]	0.69152	1.707766
Price	-0.06476	-0.044195

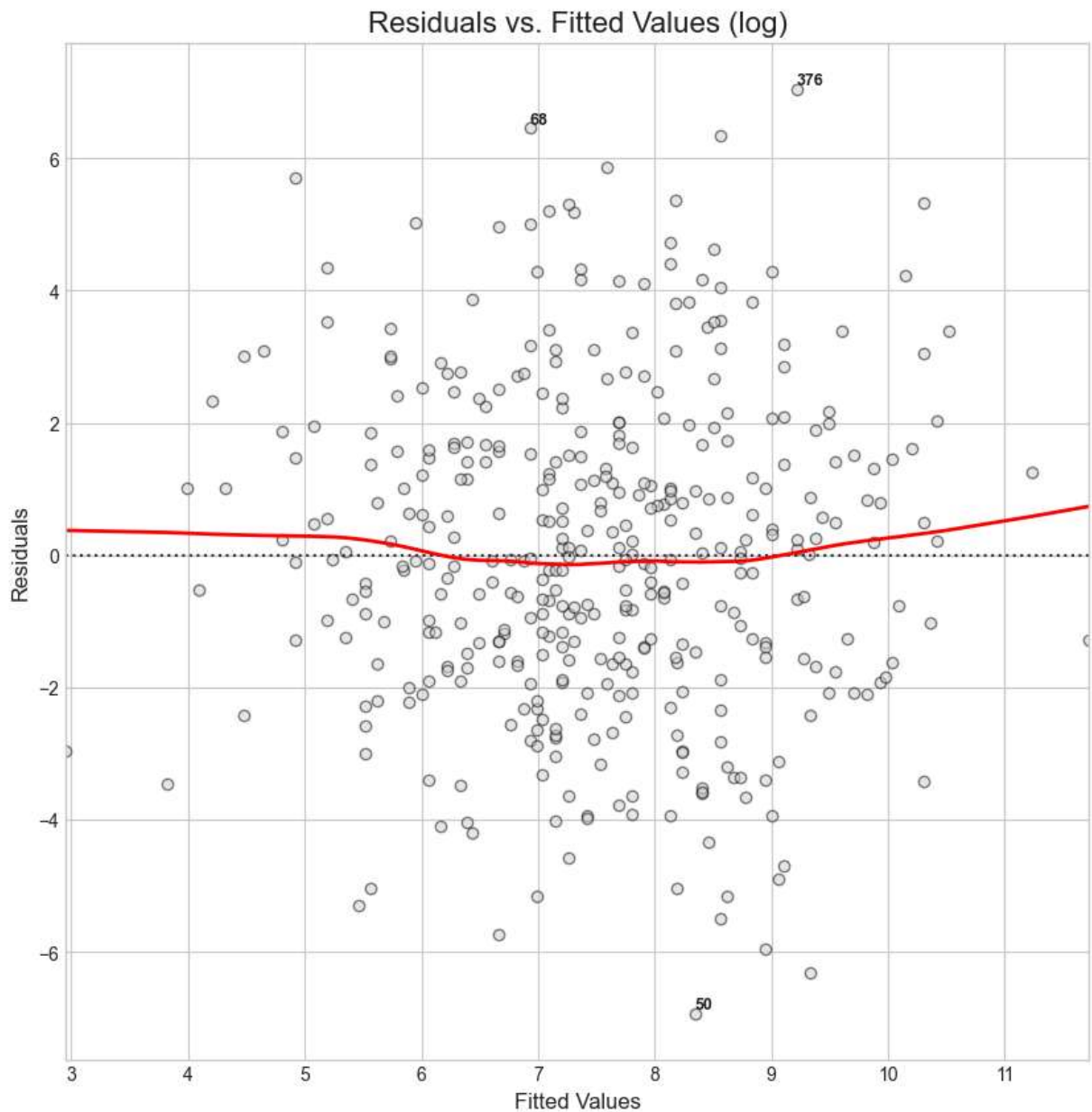
Part H

Is there evidence of outliers or high leverage observations in the model from (e)?

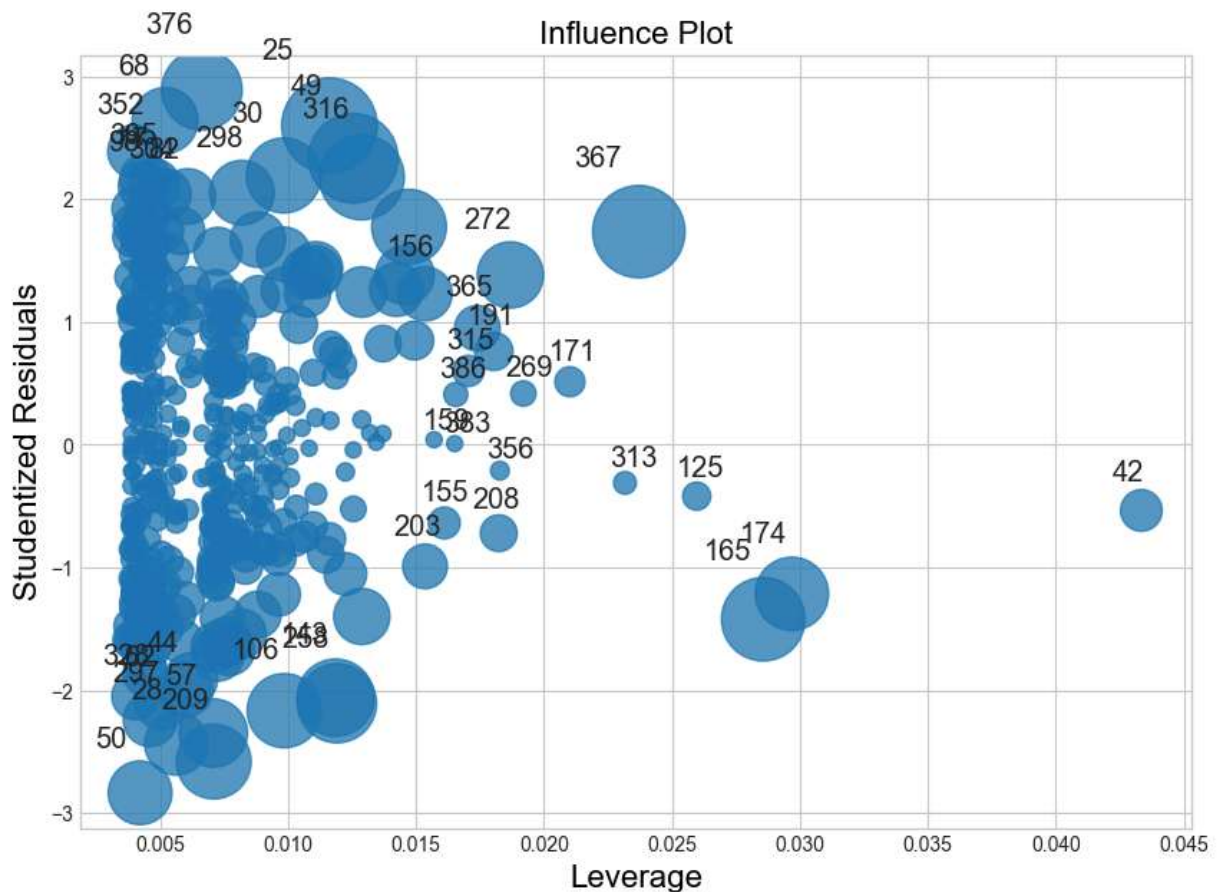
```
In [443... fig, axes = plt.subplots(1,1, figsize=(10,10));

title='Residuals vs. Fitted Values'
x_label='Fitted Values'
y_label='Residuals'

plot_residuals(smaller_carseat_model, axes, title=f'{title} (log)', x_label=x_label
```



```
In [444... fig, ax = plt.subplots(figsize=(10, 7))
fig = sm.graphics.influence_plot(smaller_carseat_model, ax=ax, criterion="cooks")
```



Observation 42 is a high leverage data point, and observations 50, 68, and 376 are outliers. Each of the outliers have a leverage less than ~0.0075, which, on top of these observations being outliers in the residuals plot, is a clear indication that they are truly outliers and do not have influence on the model.

Problem 14

This problem focuses on the collinearity problem.

Part A

Perform the following commands in Python:

```
rng = np.random.default_rng(10)
x1 = rng.uniform(0, 1, size=100)
x2 = 0.5 * x1 + rng.normal(size=100) / 10
y = 2 + 2 * x1 + 0.3 * x2 + rng.normal(size=100)
```

The last line corresponds to creating a linear model in which y is a function of x_1 and x_2 .

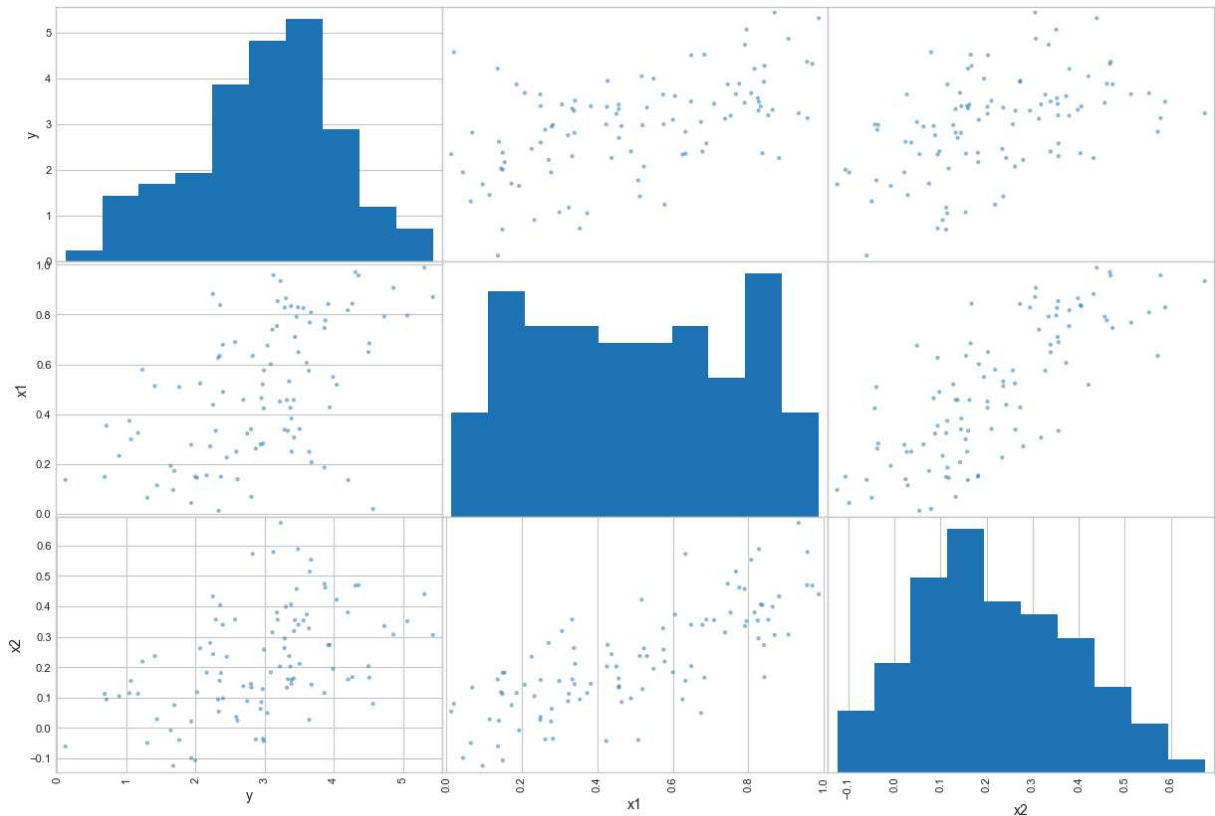
Write out the form of the linear model. What are the regression coefficients?

```
In [445... rng = np.random.default_rng(10)
x1 = rng.uniform(0, 1, size=100)
```

```
x2 = 0.5 * x1 + rng.normal(size=100) / 10
y = 2 + 2 * x1 + 0.3 * x2 + rng.normal(size=100)
```

In [446...

```
df = pd.DataFrame({'y':list(y), 'x1':list(x1), 'x2':list(x2)})
pd.plotting.scatter_matrix(df, figsize=(15,10));
```



From the linear model equation, below, the form of the linear model can be determined.

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_k x_k + \varepsilon$$

Where:

β_0 is the intercept (2),

β_1 is the regression coefficient of x_1 (2),

β_2 is the regression coefficient of x_2 (0.3),

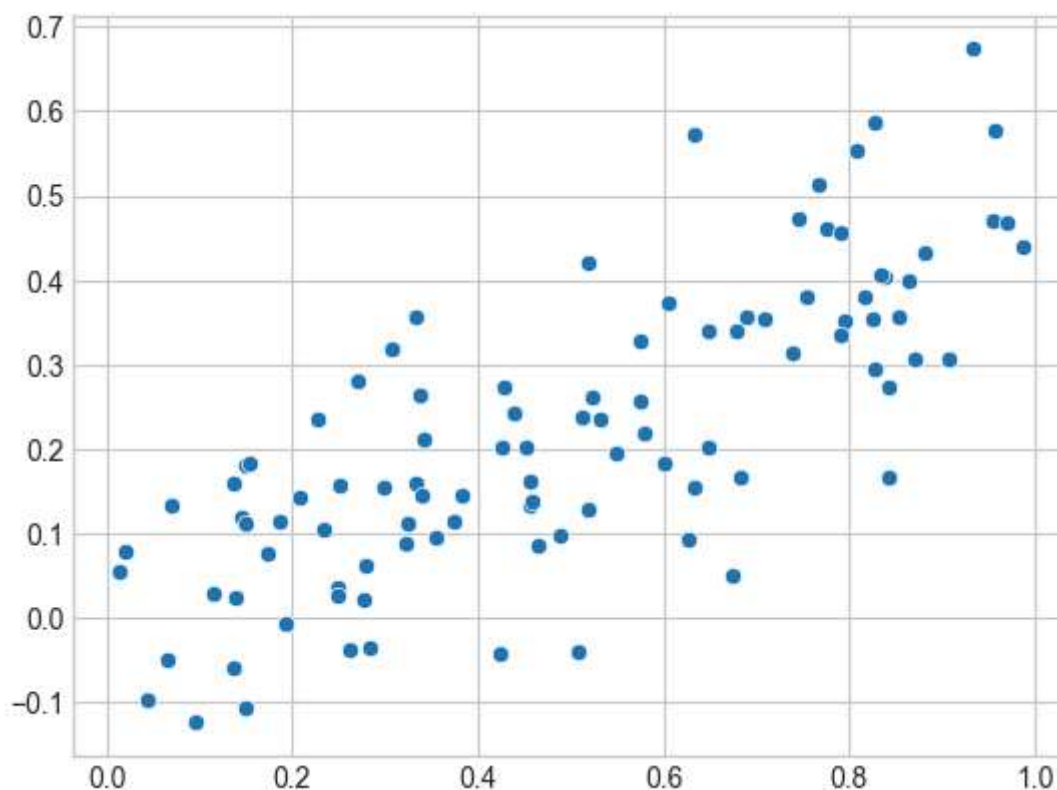
ε is the error (a randomly generated, normally distributed array of 100 observations, in this case.)

Part B

What is the correlation between x_1 and x_2 ? Create a scatterplot displaying the relationship between the variables.

```
In [447... sns.scatterplot(x=x1, y=x2)
```

```
Out[447... <Axes: >
```



```
In [448... np.corrcoef(x1, x2)
```

```
Out[448... array([[1.          , 0.7723245],
        [0.7723245, 1.          ]])
```

There is a linear relationship between these two variables, with a correlation coefficient of 0.772. A perfect correlation would have a coefficient of 1, therefore this is a strong linear correlation.

Part C

Using this data, fit a least squares regression to predict y using x_1 and x_2 . Describe the results obtained. What are $\hat{\beta}_0$, $\hat{\beta}_1$, and $\hat{\beta}_2$? How do these relate to the true β_0 , β_1 , and β_2 ? Can you reject the null hypothesis $H_0 : \beta_1 = 0$? How about the null hypothesis $H_0 : \beta_2 = 0$?

```
In [449... y = df['y']
predictors = ['x1', 'x2']
X = sm.add_constant(df[predictors])
ls_model = sm.OLS(endog=y, exog=X).fit()
print(ls_model.summary())
```

OLS Regression Results

=====						
Dep. Variable:	y	R-squared:	0.291			
Model:	OLS	Adj. R-squared:	0.276			
Method:	Least Squares	F-statistic:	19.89			
Date:	Fri, 18 Jul 2025	Prob (F-statistic):	5.76e-08			
Time:	17:06:10	Log-Likelihood:	-130.62			
No. Observations:	100	AIC:	267.2			
Df Residuals:	97	BIC:	275.1			
Df Model:	2					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

const	1.9579	0.190	10.319	0.000	1.581	2.334
x1	1.6154	0.527	3.065	0.003	0.569	2.661
x2	0.9428	0.831	1.134	0.259	-0.707	2.592
=====						
Omnibus:	0.051	Durbin-Watson:	1.964			
Prob(Omnibus):	0.975	Jarque-Bera (JB):	0.041			
Skew:	-0.036	Prob(JB):	0.979			
Kurtosis:	2.931	Cond. No.	11.9			
=====						

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

The results of the model produced the following values:

1. $\hat{\beta}_0 = 1.9579$
2. $\hat{\beta}_1 = 1.6154$
3. $\hat{\beta}_2 = 0.9428$

The values of β_0 , β_1 , and β_2 are 2, 2, and 0.3, respectively, which differ from $\hat{\beta}_0$, $\hat{\beta}_1$, and $\hat{\beta}_2$. Of the OLS predicted β values, β_1 can be rejected (p-value < 0.05), however, β_2 cannot be rejected (p-value > 0.05). This is expected, given the difference in the predicted β and the true β that were used to generate y against x1 and x2.

Part D

Now fit a least squares regression to predict y using only x1. Comment on your results. Can you reject the null hypothesis $H_0: \beta_1 = 0$?

```
In [450... y = df['y']
predictors = ['x1']
X = sm.add_constant(df[predictors])
ls_model2 = sm.OLS(endog=y, exog=X).fit()
print(ls_model2.summary())
```


OLS Regression Results

=====						
Dep. Variable:	y	R-squared:	0.281			
Model:	OLS	Adj. R-squared:	0.274			
Method:	Least Squares	F-statistic:	38.39			
Date:	Fri, 18 Jul 2025	Prob (F-statistic):	1.37e-08			
Time:	17:06:10	Log-Likelihood:	-131.28			
No. Observations:	100	AIC:	266.6			
Df Residuals:	98	BIC:	271.8			
Df Model:	1					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

const	1.9371	0.189	10.242	0.000	1.562	2.312
x1	2.0771	0.335	6.196	0.000	1.412	2.742
=====						
Omnibus:	0.204	Durbin-Watson:	1.931			
Prob(Omnibus):	0.903	Jarque-Bera (JB):	0.042			
Skew:	-0.046	Prob(JB):	0.979			
Kurtosis:	3.038	Cond. No.	4.65			
=====						

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

The reduction in R^2 indicates that this model predicts 1% less of the variance in the data, as compared to the model in Part C. The p-value for $\hat{\beta}_1$, however, has decreased to 0, and $\hat{\beta}_1$ (2.0771) is now nearly identical to the true coefficient of 2. The null hypothesis can be rejected.

Part E

Now fit a least squares regression to predict y using only x2. Comment on your results. Can you reject the null hypothesis $H_0: \beta_1 = 0$?

```
In [451... y = df['y']
predictors = ['x2']
X = sm.add_constant(df[predictors])
ls_model3 = sm.OLS(endog=y, exog=X).fit()
print(ls_model3.summary())
```

```

=====
                        OLS Regression Results
=====
Dep. Variable:          y      R-squared:          0.222
Model:                  OLS    Adj. R-squared:      0.214
Method:                 Least Squares    F-statistic:      27.99
Date:                   Fri, 18 Jul 2025    Prob (F-statistic): 7.43e-07
Time:                   17:06:10    Log-Likelihood:    -135.24
No. Observations:       100    AIC:              274.5
Df Residuals:           98    BIC:              279.7
Df Model:               1
Covariance Type:        nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	2.3239	0.154	15.124	0.000	2.019	2.629
x2	2.9103	0.550	5.291	0.000	1.819	4.002

```

=====
Omnibus:                0.191    Durbin-Watson:      1.943
Prob(Omnibus):           0.909    Jarque-Bera (JB):    0.373
Skew:                    -0.034    Prob(JB):            0.830
Kurtosis:                2.709    Cond. No.             6.11
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

This OLS model now produces a β_1 for the x2 variable that rejects the null hypothesis due to the p-value of 0. What this tells me is that there is an interaction between x1 and x2 that makes x2 a poor predictor of y when x1 is present. Referencing the scatter matrix at the beginning of this problem and the correlation coefficient in the cell, below, the coefficient of x2 in this model begins to make sense. Independently, x2 has a correlation coefficient of 0.47 with y. Ultimately, this model is a poor predictor of y, with an R^2 of 0.222.

```

In [452... np.corrcoef(y, x2)

Out[452... array([[1.          , 0.47135592],
          [0.47135592, 1.          ]])

```

Part F

Do the results obtained in (c)–(e) contradict each other? Explain your answer.

There is a contradiction in parts c-e, where β_2 in the original model does not reject the null hypothesis, but when x2 is modeled independently against y, β_1 does reject the null hypothesis. As stated in my assessment during Part E, I suspect there is an interaction between x1 and x2. Digging deeper into this idea and referencing the original equation for x2, this hypothesis is supported. The x2 data is fundamentally a transformation of x1, which, when x1 is present, would be 'washed out' by the

response between x_1 and y . Additionally, the coefficients in the equation for y further amplify the input of x_1 over x_2 , where x_1 has a coefficient of 2 and x_2 has a coefficient of 0.3. This effectively 'drowns out' the signal of x_2 in y when x_1 is present. When x_2 predicts y without the influence of x_1 , it can do so reliably, because it is a derivative of x_1 , making the model a single predictor, single response equation.

Part G

Suppose we obtain one additional observation, which was unfortunately mismeasured. We use the function `np.concatenate()` to add this additional observation to each of x_1 , x_2 and y .

```
x1 = np.concatenate([x1, [0.1]])
x2 = np.concatenate([x2, [0.8]])
y = np.concatenate([y, [6]])
```

Re-fit the linear models from (c) to (e) using this new data. What effect does this new observation have on the each of the models? In each model, is this observation an outlier? A high-leverage point? Both? Explain your answers.

In [453...

```
x1 = np.concatenate([x1, [0.1]])
x2 = np.concatenate([x2, [0.8]])
y = np.concatenate([y, [6]])

df = pd.DataFrame({'y':list(y), 'x1':list(x1), 'x2':list(x2)})
```

In [454...

```
#Model from part c
y = df['y']
predictors = ['x1', 'x2']
X = sm.add_constant(df[predictors])
ls_model = sm.OLS(endog=y, exog=X).fit()
print(ls_model.summary())
```

OLS Regression Results

=====						
Dep. Variable:	y	R-squared:	0.292			
Model:	OLS	Adj. R-squared:	0.277			
Method:	Least Squares	F-statistic:	20.17			
Date:	Fri, 18 Jul 2025	Prob (F-statistic):	4.60e-08			
Time:	17:06:11	Log-Likelihood:	-135.30			
No. Observations:	101	AIC:	276.6			
Df Residuals:	98	BIC:	284.5			
Df Model:	2					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

const	2.0618	0.192	10.720	0.000	1.680	2.443
x1	0.8575	0.466	1.838	0.069	-0.068	1.783
x2	2.2663	0.705	3.216	0.002	0.868	3.665
=====						
Omnibus:	0.139	Durbin-Watson:	1.894			
Prob(Omnibus):	0.933	Jarque-Bera (JB):	0.320			
Skew:	0.013	Prob(JB):	0.852			
Kurtosis:	2.725	Cond. No.	9.68			
=====						

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
In [455... # Model from part D
y = df['y']
predictors = ['x1']
X = sm.add_constant(df[predictors])
ls_model2 = sm.OLS(endog=y, exog=X).fit()
print(ls_model2.summary())
```

OLS Regression Results

=====						
Dep. Variable:	y	R-squared:	0.217			
Model:	OLS	Adj. R-squared:	0.209			
Method:	Least Squares	F-statistic:	27.42			
Date:	Fri, 18 Jul 2025	Prob (F-statistic):	9.23e-07			
Time:	17:06:11	Log-Likelihood:	-140.37			
No. Observations:	101	AIC:	284.7			
Df Residuals:	99	BIC:	290.0			
Df Model:	1					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

const	2.0739	0.201	10.310	0.000	1.675	2.473
x1	1.8760	0.358	5.236	0.000	1.165	2.587
=====						
Omnibus:	8.232	Durbin-Watson:		1.636		
Prob(Omnibus):	0.016	Jarque-Bera (JB):		10.781		
Skew:	0.396	Prob(JB):		0.00456		
Kurtosis:	4.391	Cond. No.		4.61		
=====						

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
In [456... # Model from part E
y = df['y']
predictors = ['x2']
X = sm.add_constant(df[predictors])
ls_model3 = sm.OLS(endog=y, exog=X).fit()
print(ls_model3.summary())
```

OLS Regression Results

=====						
Dep. Variable:	y	R-squared:	0.267			
Model:	OLS	Adj. R-squared:	0.260			
Method:	Least Squares	F-statistic:	36.10			
Date:	Fri, 18 Jul 2025	Prob (F-statistic):	3.13e-08			
Time:	17:06:11	Log-Likelihood:	-137.01			
No. Observations:	101	AIC:	278.0			
Df Residuals:	99	BIC:	283.3			
Df Model:	1					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

const	2.2840	0.151	15.088	0.000	1.984	2.584
x2	3.1458	0.524	6.008	0.000	2.107	4.185
=====						
Omnibus:	0.495	Durbin-Watson:	1.939			
Prob(Omnibus):	0.781	Jarque-Bera (JB):	0.631			
Skew:	-0.041	Prob(JB):	0.729			
Kurtosis:	2.621	Cond. No.	5.84			
=====						

Notes:

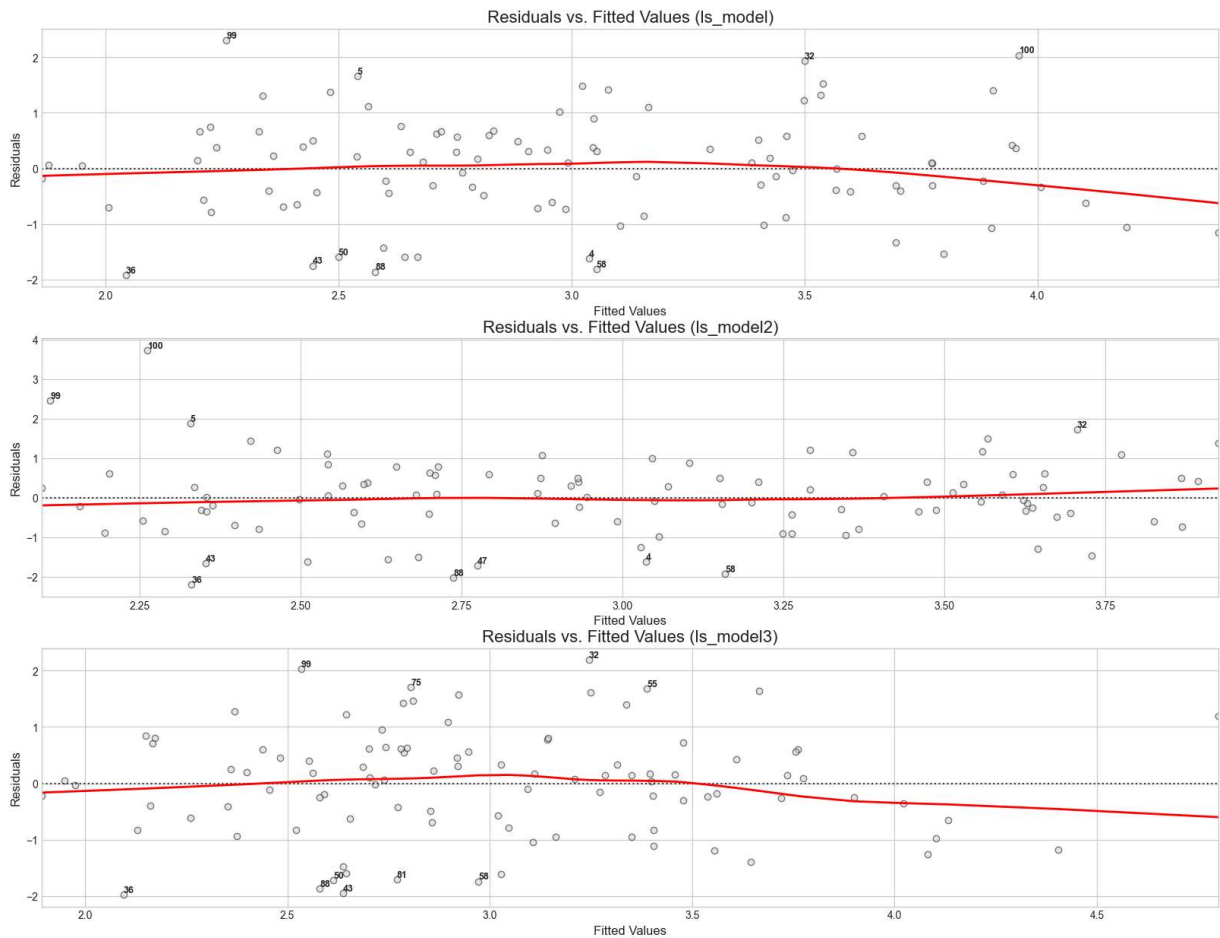
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Based on the impact on each of the models, this data point is a high leverage point. In `ls_model`, `x1` is no longer a statistically significant predictor of `y`, instead, `x2` is. This is a complete reversal from the initial dataset. Additionally, the R^2 value has increased to 0.316 from 0.291 and β_1 and β_2 have changed from 1.6154 and 0.9428 to 0.4907 and 2.9070, respectively. In `lm_model2`, `x1` is still a statistically significant predictor of `y`, but the R^2 value has decreased to 0.167. In `lm_model3`, `x2` is still a statistically significant predictor of `y`, but the R^2 value has increased to 0.307. To further verify my analysis, let's take a look at the residual and leverage plots.

```
In [457... fig, axes = plt.subplots(3,1, figsize=(20,15));

plot_residuals(ls_model, axes[0], title=f'{title} (ls_model)', x_label=x_label, y_label=y_label)
plot_residuals(ls_model2, axes[1], title=f'{title} (ls_model2)', x_label=x_label, y_label=y_label)
plot_residuals(ls_model3, axes[2], title=f'{title} (ls_model3)', x_label=x_label, y_label=y_label)

plt.show()
```

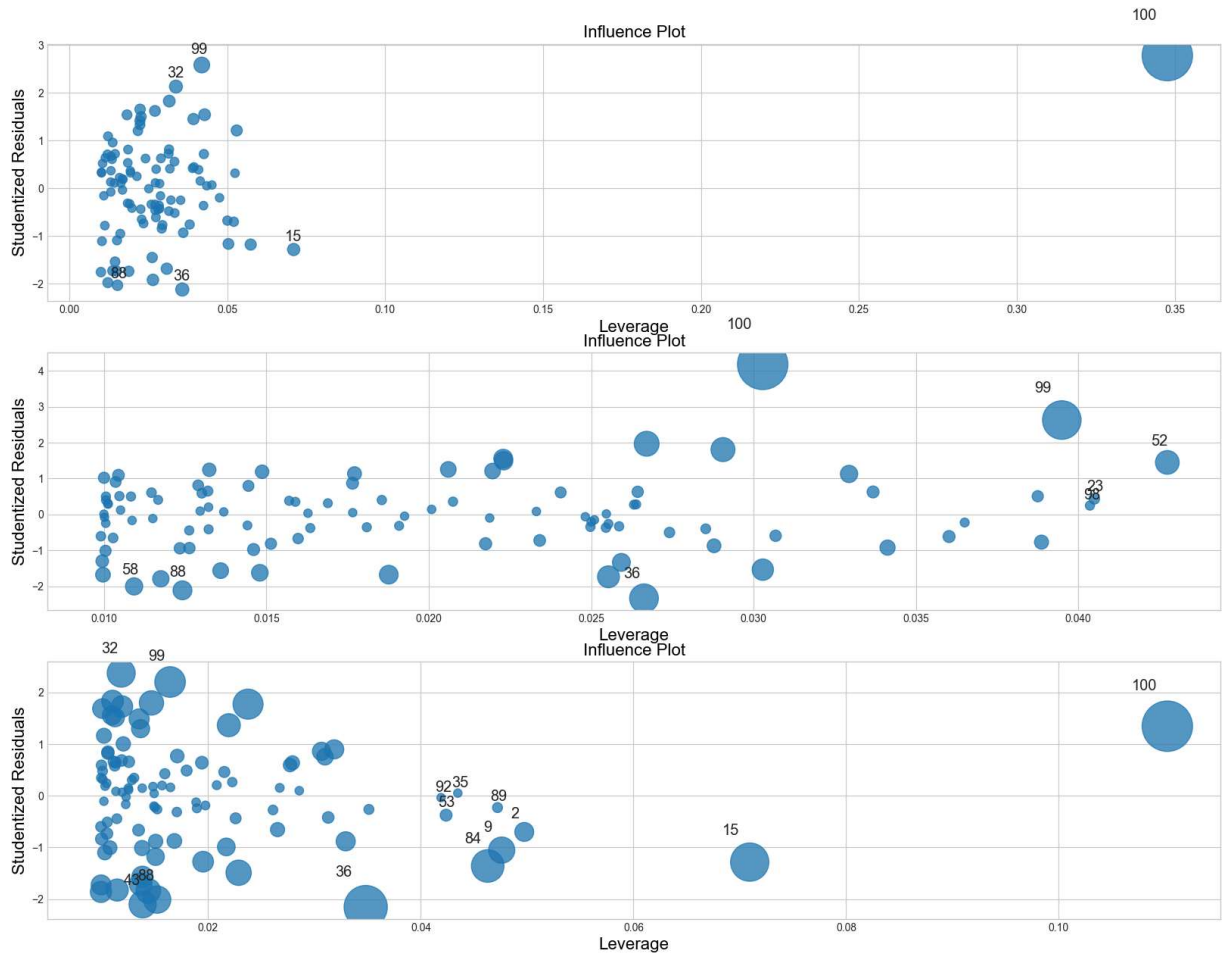



In [458...

```
fig, axes = plt.subplots(3,1, figsize=(20,15));

sm.graphics.influence_plot(ls_model1, ax=axes[0], criterion="cooks")
sm.graphics.influence_plot(ls_model2, ax=axes[1], criterion="cooks")
sm.graphics.influence_plot(ls_model3, ax=axes[2], criterion="cooks")

plt.show()
```



As I had expected, the additional observation (101) has a leverage of ~ 0.27 and ~ 0.097 in `ls_model` and `ls_model3`, respectively. In `ls_model2`, this observation is an outlier, which is indicative of this observation being proportionally more influential on x_2 . This is also supported by the leverage plots for the respective variables.